Quantum Approximate Optimization Algorithm and The Ising Model

Reading Project (sem 6)
Final Report

submitted by
Ashis Nayak
School of Physical Sciences
UM-DAE-CEBS, Mumbai

Under the Guidance of
Prof. Anuradha Misra
School of Physical Sciences
UM-DAE-CEBS, Mumbai

Date:12/05/2026

CERTIFICATE

This is to certify that the reading project entitled **"Quantum Approximate Optimization Algorithm and The Ising Model"** has been carried out by **Mr Ashis Nayak**, Roll Number P0231748, School of Physical Sciences, UM-DAE-CEBS Mumbai, under my guidance.

The work presented in this report is based on the study, discussions, and reading carried out during the course of the project.

Date: 15th May 2026

Prof. Anuradha Misra
CSIR Emeritus Scientist
School of Physical Sciences
UM-DAE-CEBS Mumbai

Acknowledgement

I would like to express my sincere gratitude to Dr. Anuradha Misra, UM DAE Centre for Excellence in Basic Sciences, for her guidance and support throughout this semester for the reading project. Her guidance and encouragement at every stage were invaluable and helped me achieve my goal of understanding and delving deep into this topic.

I am especially thankful for her patience in addressing my questions and for continuously motivating me to think critically and explore concepts more deeply. This project has been a highly enriching experience, and I am grateful for the opportunity to work under her supervision. I also extend my thanks to my peers and seniors who were always present to answer my questions , discuss and rectify my doubts. I would like to thank my senior A.R Bathri Narayan and my peer U Manasvi Maiya and the students of University of Mumbai who were present for my presentations and discussions which added a lot of value and input that went into this report.

Abstract

This report gives a brief introduction to the Quantum Approximate Optimization Algorithm (QAOA) and its connection with machine learning, the Ising model, and combinatorial optimization. It explains how optimization problems can be represented using Ising and QUBO formulations, and how ideas from adiabatic quantum computing lead naturally to the QAOA framework.

The Max-Cut problem is used as the main example to show how a graph optimization problem can be encoded into a cost Hamiltonian and solved using a variational quantum circuit. A small Qiskit simulation is also discussed to illustrate the working of the algorithm. Overall, the report aims to provide a concise conceptual and mathematical understanding of QAOA and its relevance to quantum optimization.

Contents

Chapter 1

Introduction

Quantum Approximate Optimization Algorithm (QAOA) is one of the most important variational quantum algorithms proposed for solving combinatorial optimization problems on near term quantum devices. Its significance lies in the fact that many practically relevant tasks in science, engineering, and decision-making can be formulated as optimization problems over discrete variables, where the objective is to identify the best configuration among a very large number of possibilities. Examples include graph partitioning, resource allocation, scheduling, routing, and network design. The study of QAOA is therefore valuable not only from the perspective of quantum computation, but also for understanding how physical models and algorithmic strategies can be used to approach complex real-world optimization tasks.

This report focuses on the conceptual and mathematical foundations of QAOA through its connection to the Ising model, adiabatic evolution, and graph-based optimization. The scope of the report is primarily introductory and analytical. It aims to explain how a combinatorial optimization problem can be translated into a Hamiltonian description, how such a Hamiltonian can be used in a quantum algorithmic setting, and how QAOA emerges as a discretized and variational alternative to adiabatic quantum optimization. Rather than attempting to establish a large-scale computational advantage, the report is designed to build a coherent understanding of the method and to illustrate its working through small simulated examples. In this sense, the report serves as both a theoretical introduction and a problem-oriented case study in quantum optimization.

To develop this perspective, the report begins with a short background discussion on optimization-related concepts from machine learning and data representation, which help motivate why objective functions, similarity measures, and structured search problems are important in modern computational science. It then introduces the Ising model as a bridge between statistical physics and optimization, showing how spin configurations may be interpreted as binary decision variables. The relation between the Ising formalism and QUBO problems is discussed in order to make clear why many discrete optimization problems can be encoded in a form suitable for quantum computation. This is followed by a discussion of time evolution, transverse fields, adiabatic quantum computing, and quantum annealing, which collectively motivate the variational framework used in QAOA.

The report then applies these ideas to the Max-Cut problem, which serves as the principal example throughout the study. Max-Cut is a natural benchmark because it is simple to formulate, graph theoretically meaningful, and directly expressible in terms of an Ising-type cost Hamiltonian. The final part of the report presents a Qiskit-based simulation for two four node graph instances, allowing the algorithm to be studied through explicit circuit construction,

classical parameter optimization, and measurement statistics. These simulations show that the probability distribution becomes concentrated on the bit strings corresponding to the optimal cuts of the graphs considered. In the first case, the algorithm identifies the two alternating bit strings associated with the maximum cut of the four edge square graph, while in the second case it captures the six optimal configurations of the denser six edge graph. The reported approximation ratios are close to unity, indicating that the optimized circuits produce expectation values very near the true optimum for these small examples.

Overall, the main finding of the report is that QAOA provides a clear and physically motivated framework for translating combinatorial optimization problems into quantum circuits whose parameters can be improved by a classical optimization loop. Even though the examples studied are small and simulated, they demonstrate the essential workflow of the method and highlight why QAOA is considered a promising approach for constrained optimization problems such as network design, scheduling, allocation, and related graph-structured tasks. The report therefore aims to show that QAOA is not merely a formal construction, but a practically meaningful algorithmic framework situated at the intersection of quantum physics, optimization theory, and computational problem solving.

Chapter 2

Introduction To Machine Learning

2.1 Machine Learning

Machine learning is a branch of artificial intelligence concerned with developing algorithms that learn patterns from data and use those patterns to make predictions, classifications, or decisions without being explicitly programmed for every individual task. Instead of following only fixed rules, a machine learning model improves its performance by identifying structure in past observations and applying that knowledge to new inputs.

2.2 Types of Machine Learning Algorithms

Machine learning algorithms can be organized according to the kind of task they solve. Some algorithms predict a continuous numerical quantity, some assign an input to one of a few discrete categories, and others try to discover structure in unlabeled data. In this section, we briefly discuss a few foundational problems and the methods commonly used to address them [1].

2.2.1 Linear Regression

Linear regression is a supervised learning algorithm used to model the relationship between input variables and a continuous output variable. The model assumes that the prediction can be written as a linear combination of the input features.

Suppose the input is represented by $x = (x_1, x_2, \ldots, x_n)$. Then a linear regression model predicts the target value as

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b, \tag{2.1}$$

where w_i are the model parameters and b is the bias term.

- It is mainly used when the target variable is continuous, such as temperature, stock value, or house price.
- Training usually involves minimizing the mean squared error between the predicted values and the true values.
- The learned coefficients provide an interpretable measure of how each feature influences the output.

- Although simple, linear regression often serves as a baseline model for more complicated regression tasks.

2.2.2 Binary Classification

Binary classification is a supervised learning problem in which each data point must be assigned to one of two possible classes. Typical examples include spam or not spam, diseased or healthy, and accepted or rejected.

- The output is discrete, usually encoded as 0 and 1.
- Binary classification differs from regression because the goal is not to predict a continuous value but to assign a label.
- Performance is commonly evaluated using accuracy, precision, recall, F1-score, and the ROC curve (Receiver Operating Characteristic curve)[1].
- Many algorithms can solve binary classification problems, including logistic regression, support vector machines, decision trees, and neural networks.

2.2.3 Clustering Problem

Clustering is an unsupervised learning problem in which the aim is to divide a dataset into groups, called clusters, such that points within the same cluster are more similar to each other than to points in other clusters.

Unlike supervised learning, clustering does not require labeled examples. Instead, the algorithm tries to infer the hidden structure of the data directly from the features. One of the most widely used methods is K-means clustering, which partitions the data into K clusters by iteratively updating cluster centers and assigning points to the nearest center.

- Clustering is useful for exploratory data analysis, pattern discovery, image segmentation, and customer grouping.
- The number of clusters may be chosen in advance or estimated using heuristic methods such as the elbow method or silhouette score.
- Since there are no ground-truth labels, evaluating clustering quality is generally more difficult than evaluating supervised tasks.
- Common clustering algorithms include K-means, hierarchical clustering, and grover based K-means.

[1]The F1-score is the harmonic mean of precision and recall, and is useful when both false positives and false negatives must be considered. The ROC curve, or Receiver Operating Characteristic curve, plots the true positive rate against the false positive rate at different classification thresholds.

2.2.4 Principal Component Analysis (PCA)

Principal Component Analysis is an unsupervised dimensionality reduction technique used to transform a dataset with many correlated variables into a new set of orthogonal variables called principal components.

The first principal component is the direction along which the data has the largest variance. The second principal component is orthogonal to the first and captures the next largest variance, and so on. By keeping only the first few components, one can represent the data in a lower-dimensional space while preserving most of its important variation.

- PCA is often used for visualization, noise reduction, feature extraction, and data compression.
- It is especially useful when the original features are highly correlated.
- The principal components are obtained from the eigenvectors of the covariance matrix, or equivalently from singular value decomposition.
- PCA is not itself a classifier or regressor; rather, it is a preprocessing tool that can improve the performance of other learning algorithms.

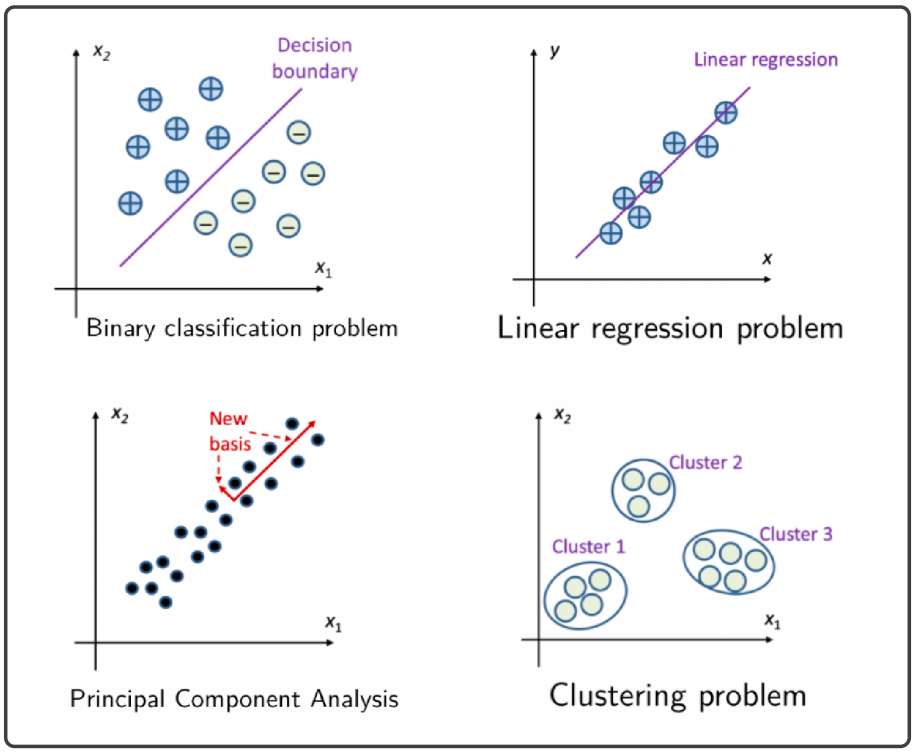


Figure 2.1: Different type of data set and the techniques used for applying Machine Learning on them

Source: Fig 14.2, 14.4, 14.6 & 14.7 from [1].

2.3 Important Terminologies

The following are some important terminologies used in Machine Learning and their definitions [1]:

2.3.1 Feature Vector

A feature vector is the mathematical representation of a data point in terms of its measured attributes. If a dataset has d features, then each sample can be represented as a vector in a d-dimensional space:

$$x_j = \sum_{i=1}^{d} x_{ij} e_i = \begin{bmatrix} x_{1j} \\ x_{2j} \\ \vdots \\ x_{dj} \end{bmatrix},$$

where the basis vector e_i corresponds to the ith feature and j labels the data instance. This representation is central in machine learning because most learning algorithms operate on vectors of this form. Once the data are written as feature vectors, one can compare samples, compute distances, and define predictive models in a systematic way.

2.3.2 Cosine Similarity

The cosine similarity between two vectors is

$$\cos(x_j, x_k) = \frac{x_j^T x_k}{\|x_j\| \, \|x_k\|}.$$

It measures the alignment between two vectors and is useful when direction is more important than magnitude. A cosine similarity close to 1 indicates that two samples point in nearly the same direction and are therefore similar in pattern, whereas a value close to 0 indicates weak similarity. This measure is commonly used in text analysis, information retrieval, and high-dimensional datasets where the overall scale of the vector is less important than the relative composition of features.

2.3.3 Hamming Distance

For binary-valued data, the Hamming distance is defined as

$$d_H(x_j, x_k) = \sum_{i=1}^{d} x_{ij} \oplus x_{ik}, \quad \text{where } \oplus \text{ is addition modulo 2}$$

It counts the number of positions at which two binary strings differ. Therefore, it is a natural measure of dissimilarity for bit strings, binary codes, and discrete spin-like variables. In problems involving binary data, the Hamming distance provides a simple way to quantify how many feature values must change in order to transform one configuration into another.

2.4 Loss Functions and Generalization

The loss function $L(y, f(x))$ measures the discrepancy between the true output y and the predicted output $f(x)$. It is one of the key ingredients of machine learning because it tells us how well a model performs on individual examples. During training, model parameters are adjusted so as to reduce the overall loss over the dataset.

2.4.1 Common Loss Functions

Two standard choices for regression are:

- **Squared error:**
$$L(y_j, f(x_j)) = [y_j - f(x_j)]^2$$
This loss penalizes large errors more strongly because the deviation is squared, which makes it especially useful when large mistakes should be discouraged.

- **Absolute error:**
$$L(y_j, f(x_j)) = |y_j - f(x_j)|$$
This loss grows linearly with the prediction error and is often more robust to outliers than squared error.

These loss functions are widely used in regression problems where the target variable is continuous.

2.4.2 Empirical Risk

The average loss on the training dataset is called the empirical risk or training error:

$$E = \frac{1}{n} \sum_{j=1}^{n} L(y_j, f(x_j)).$$

It summarizes how well the model fits the observed training samples. Minimizing the empirical risk is the basic objective in many machine learning methods. However, a model that achieves a very small training error is not necessarily useful if it simply memorizes the training data.

2.4.3 Generalization Error

The generalization error measures performance on previously unseen data drawn from the underlying distribution:

$$E_{GE} = \int_{x \in X,\, y \in Y} L(y, f(x)) p(y, x)\, dx\, dy.$$

This quantity is more important than the training error because the real aim of machine learning is to perform well on new examples rather than only on the data used for training. In practice, since $p(y, x)$ is usually unknown, generalization is estimated using validation or test data. A good model should achieve low training error and also generalize well, thereby avoiding both underfitting and overfitting.

Chapter 3

The Ising Model

3.1 Introduction to the Ising Model

The Ising model is one of the simplest and most important models in statistical physics for studying collective behavior in systems of interacting magnetic moments. It was originally introduced to understand ferromagnetism, that is, the tendency of microscopic magnetic dipoles inside a material to align in a common direction. Despite its simple mathematical structure, the model captures essential physical ideas such as ordering, thermal disorder, and phase transitions, and for this reason it has become a standard framework in both physics and optimization theory.

In the Ising model, each spin variable takes one of two possible values, $\sigma_i \in \{-1, 1\}$, which may be interpreted as an "up" spin or a "down" spin. These spins are placed on the vertices of a graph, most commonly on a one-dimensional chain or a two-dimensional lattice, and each spin interacts primarily with its nearest neighbors. The system may also be influenced by an external magnetic field, which tends to bias the spins toward a preferred orientation. When neighboring spins align, the interaction energy is lowered, so the system energetically favors ordered configurations. On the other hand, thermal fluctuations arising from finite temperature tend to randomize the spin orientations and disrupt this order. The balance between these competing effects, interaction driven ordering and temperature driven disorder determines the macroscopic behavior of the material. As temperature or external parameters are varied, the model can exhibit phase transitions, where the qualitative behavior of the system changes abruptly, for example from a magnetized ordered phase to a disordered paramagnetic phase.

3.2 Hamiltonian Formulation

The Hamiltonian of the Ising model is

$$H(\sigma) = -\sum_{\langle i,j \rangle} J_{ij}\sigma_i\sigma_j - \mu \sum_i h_i\sigma_i.$$

Here, J_{ij} denotes the interaction strength between the ith and jth sites, h_i denotes the strength of the external magnetic field acting on the ith spin site, μ is the magnetic moment, and $\langle i, j \rangle$ denotes nearest-neighbor pairs.

The sign of J_{ij} determines the type of interaction:

- $J_{ij} > 0$: ferromagnetic interaction,
- $J_{ij} < 0$: antiferromagnetic interaction,
- $J_{ij} = 0$: no interaction.

If $h_i > 0$, the ith spin tends to align with the external field in the positive direction.

3.3 Boltzmann Distribution and Optimal Configuration

The probability of a configuration is described by the Boltzmann distribution:

$$P_\beta(\sigma) = \frac{1}{Z_\beta} e^{-\beta H(\sigma)}, \qquad \beta = \frac{1}{k_B T},$$

where β is the inverse temperature and Z_β is the partition function,

$$Z_\beta = \sum_\sigma e^{-\beta H(\sigma)}.$$

If the interaction matrix is denoted by $J = (J_{ij})$, then the optimum configuration may be written as

$$\hat{\sigma} = \arg\min_\sigma (\sigma^T J \sigma + h^T \sigma).$$

This optimization viewpoint is important because it connects the Ising model to classical and quantum optimization problems. Thus it is natural to map this problem to a known optimization problem called the ***QUBO problem***.

3.4 QUBO Formulation

Quadratic unconstrained binary optimization (QUBO) is a standard way of expressing optimization problems using binary decision variables. In this framework, each variable can take only the value 0 or 1, and the aim is to find the binary string that minimizes a quadratic cost function. The term *quadratic* means that the objective includes both individual variable contributions and pairwise interaction terms, while *unconstrained* means that any conditions from the original problem are typically absorbed into the objective function through penalty terms. For this reason, many combinatorial optimization problems can be rewritten in QUBO form.

In a QUBO problem, the variables are binary, $x_i \in \{0, 1\}$, and the objective function is quadratic:

$$\hat{x} = \arg\min_x (x^T W x + b^T x).$$

The Ising problem can be reformulated as a QUBO problem by defining $\sigma_i = 2x_i - 1$, taking $\mu = 1$, and identifying $b_i = h_i$ as the bias term.

The Ising Hamiltonian can also be written in terms of Pauli operators as

$$\hat{H} = -\sum_{\langle i,j \rangle} J_{ij} \sigma_i^Z \sigma_j^Z - \sum_i h_i \sigma_i^Z,$$

where

$$\sigma^Z = Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}.$$

3.5 Time Evolution and Transverse Field

The quantum state evolves according to

$$i\hbar \frac{dU}{dt} = HU.$$

For a time-independent Hamiltonian,

$$U = e^{-iHt/\hbar}.$$

With a transverse field, the Hamiltonian becomes

$$\hat{H} = -\sum_{\langle i,j \rangle} J_{ij} \sigma_i^Z \sigma_j^Z - \sum_i h_i \sigma_i^X,$$

where

$$\sigma^X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}.$$

The transverse field introduces quantum fluctuations that allow transitions between spin configurations.
The discussion so far has established how the Ising model can be expressed in both classical and quantum language, and how its ground-state search can be interpreted as an optimization problem. This naturally leads to the next question: how can one use quantum dynamics to reach or approximate this ground state efficiently? An approximate solution is provided by adiabatic quantum computing and quantum annealing.

Chapter 4

Adiabatic Quantum Computing and Quantum Annealing

4.1 Adiabatic Principle

An adiabatic process is a process that changes slowly enough that the system can continuously adapt its configuration. If the system starts in the ground state of an initial Hamiltonian H_0, then under sufficiently slow evolution it will remain close to the instantaneous ground state and eventually end in the ground state of the final Hamiltonian H_1. This statement is the content of the adiabatic theorem, originally formulated by Born and Fock.

The time dependent Hamiltonian for this process is in general written as:

$$H(t) = (1-t)H_0 + tH_1, \qquad t \in [0,1]. \tag{4.1}$$

As the parameter t increases from 0 to 1, the system moves from the initial Hamiltonian H_0 to the target Hamiltonian H_1, provided the evolution is sufficiently slow and the ground state remains non-degenerate.

4.2 Adiabatic Evolution for the Ising Model

For the Ising model, one typically chooses the initial Hamiltonian to be

$$H_0 = -\sum_i X_i,$$

which describes the interaction of a transverse field with the spin sites. The final Hamiltonian is chosen as

$$H_1 = -\sum_{\langle i,j \rangle} J_{ij} Z_i Z_j - \sum_i h_i X_i.$$

The system is then guided from an easily prepared initial ground state toward the ground state of the problem Hamiltonian. This idea forms the basis of quantum annealing.

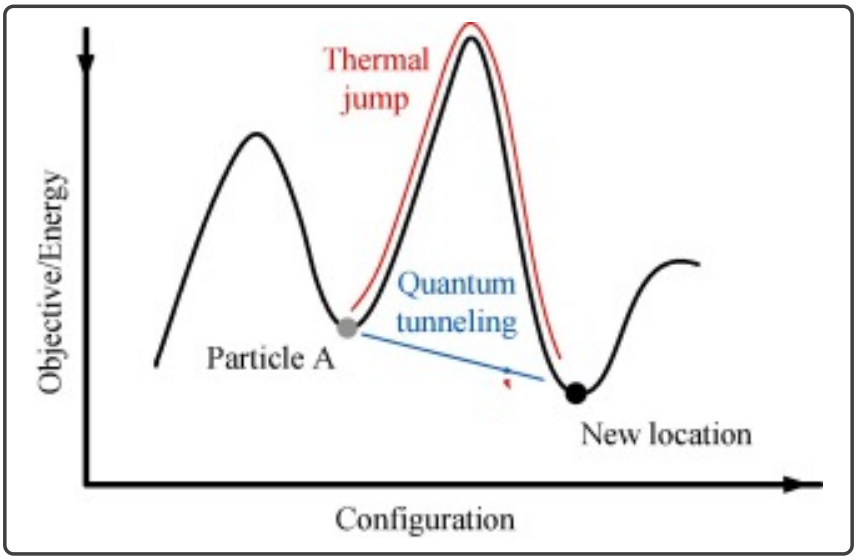


Figure 4.1: Quantum annealing [1].

The beauty of Quantum Annealing lies in the fact that quantum mechanically wavefunctions can tunnel through barriers unlike classical particles. Thus approaching a global minimum becomes easier and requires lower energy than thermal excitation or any other classical way. Quantum computers that are specifically designed to carry out such annealing tasks are called ***Quantum Annealers***.

Chapter 5

Variational Quantum Algorithms and Quantum Approximate Optimization Algorithm

5.1 VQE

Quantum circuits are inherently prone to noise and operational errors, which makes deep gate-based implementations difficult on near-term quantum hardware. This motivates the use of variational quantum circuits, which combine quantum state preparation with classical optimization. Such methods are particularly suitable for noisy and imperfect quantum computers.

One important example is the variational quantum eigensolver (VQE), which estimates the ground-state energy of a Hamiltonian by preparing a parameterized quantum state and minimizing the expectation value of the Hamiltonian with respect to the circuit parameters.

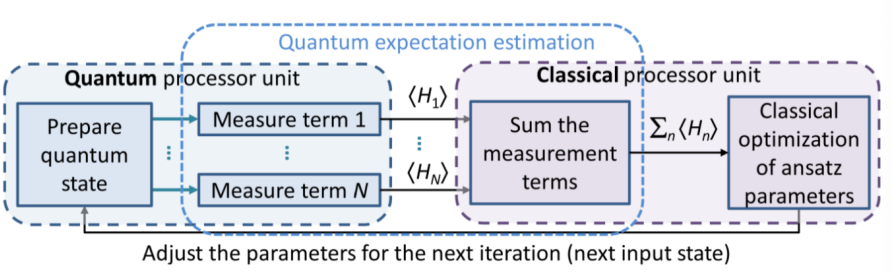


Figure 5.1: Variational quantum eigensolver [1].

5.2 QAOA

The key idea behind the Quantum Approximate Optimization Algorithm (QAOA) is to approximate the adiabatic evolution pathway by a sequence of discrete unitary operations. Let $U(t)$ denote the ideal continuous evolution operator. The final operator $U(T)$ can be approximated by a product of simpler operators if the Hamiltonian changes gradually enough.

A discretized step may be written as [1]:

$$U'_k = e^{-i\frac{T}{p}H(kT/p)} = e^{-i\frac{T}{p}(\frac{k}{p}H_f+(1-\frac{k}{p})H_0)}, \text{ Where } H_f \text{ is } H_1, \text{ the final hamiltonian.} \quad (5.1)$$

Here, the total computation time T is divided into p intervals.

Using the Campbell–Baker–Hausdorff theorem, one may approximate this by a product of separate evolutions under H_f and H_0:

$$U'_k \cong U(H_f, \tfrac{k}{p}\tfrac{T}{p})U(H_0, (1-\tfrac{k}{p})\tfrac{T}{p}). \tag{5.2}$$

More generally, the kth step can be written as

$$U'_k = U(H_f, \gamma_k)U(H_0, \beta_k), \qquad \beta_k + \gamma_k = \triangle T_k, \qquad \sum_{k=1}^{p} \triangle T_k = T. \tag{5.3}$$

This decomposition need not be uniform, which gives flexibility in optimizing the parameters.

5.3 Trotterized Approximation

Thus, one alternates the evolution under H_0 and H_f to approximate the adiabatic pathway:

$$U(T) \cong U(H_f, \gamma_p)U(H_0, \beta_p)\dots U(H_f, \gamma_1)U(H_0, \beta_1). \tag{5.4}$$

This approximation is commonly known as Trotterization or the Trotter–Suzuki decomposition.

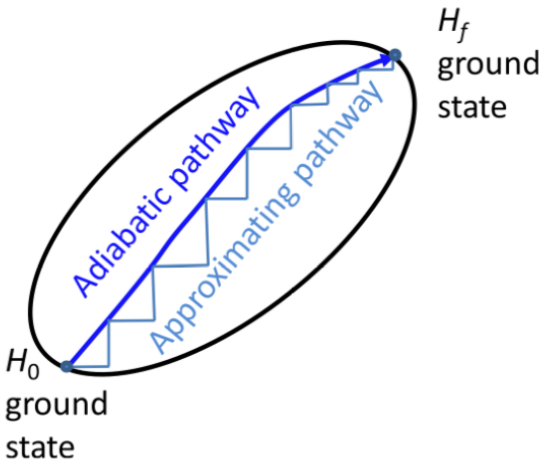


Figure 5.2: Approximation of the adiabatic pathway [1].

Because the chosen initial Hamiltonian is $H_0 = -\sum_i X_i$, its ground state is the tensor product of $|+\rangle = \frac{|0\rangle+|1\rangle}{\sqrt{2}}$ over all qubits:

$$|\psi_0\rangle = |+\rangle^{\otimes N} = \frac{1}{\sqrt{2^N}} \sum_z |z\rangle\,, \qquad z \in \{0,1\}^N. \tag{5.5}$$

The final state after p QAOA steps is then

$$|\psi_f(\beta,\gamma)\rangle = |\beta,\gamma\rangle = U(H_f, \gamma_p)U(H_0, \beta_p)\dots U(H_f, \gamma_1)U(H_0, \beta_1)\,|\psi_0\rangle\,. \tag{5.6}$$

5.4 Cost Operator and Expectation Value

Suppose the optimization objective is

$$C_{\max} = \max_z C(z) = \max_z \sum_{k=1}^{m} C_k(z), \qquad z \in \{0,1\}^N, \tag{5.7}$$

where each C_k encodes a Boolean or weighted constraint. The objective function can be represented by an operator C, called the *cost function*, satisfying:

$$C\,|z\rangle = \sum_{k=1}^{m} C_k(z)\,|z\rangle = f(z)\,|z\rangle\,. \tag{5.8}$$

The eigenvalues of C are therefore the values $f(z) = \sum_k C_k(z)$, and the largest eigenvalue is $C_{\max} = f(z_{\max})$ For an arbitrary state $|\psi\rangle = \sum_z \alpha_z\,|z\rangle$, the expectation value of the cost operator is bounded by the maximum value of the objective function:

$$\langle\psi|\,C\,|\psi\rangle \leq f(z_{\max}). \tag{5.9}$$

For the initial equal superposition state, one obtains

$$\langle\psi_0|\,C\,|\psi_0\rangle = \frac{1}{2^N} \sum_{z\in\{0,1\}^N} f(z). \tag{5.10}$$

The probability of obtaining the maximizing configuration $|z_{\max}\rangle$ from a single measurement of the uniform initial state is therefore only 2^{-N}

5.5 QAOA Unitaries

The corresponding unitary evolution generated by the cost operator is

$$U(C,\gamma) = e^{-i\gamma C} = \prod_{k=1}^{m} e^{-i\gamma C_k}, \tag{5.11}$$

because the terms commute when $[C_i, C_j] = 0$. On an arbitrary state,

$$U(C,\gamma)\,|\psi\rangle = \sum_z \alpha_z \prod_{k=1}^{m} e^{-i\gamma C_k}\,|z\rangle\,. \tag{5.12}$$

Thus, each basis state receives a phase depending on how many constraints it satisfies.

The evolution under the initial Hamiltonian $B = H_0 = -\sum_i X_i$ is given by

$$U(B,\beta) = e^{-i\beta H_0} = \prod_{k=1}^{N} e^{-i\beta X_k} \equiv \prod_{k=1}^{N} U(B_k,\beta). \tag{5.13}$$

Each $U(B_k,\beta)$ is simply a single-qubit rotation of the form $R_x(2\beta)$.

Using Eq. (5.6), the final QAOA state becomes

$$|\beta,\gamma\rangle = U(C,\gamma_p)U(B,\beta_p)\ldots U(C,\gamma_1)U(B,\beta_1)\,|\psi_0\rangle\,. \tag{5.14}$$

The optimization problem then becomes

$$C_p = \max_{(\beta,\gamma)} \langle \beta, \gamma | \, C \, | \beta, \gamma \rangle \, . \tag{5.15}$$

The parameters must be chosen appropriately so that the accuracy improves as p increases, ideally satisfying

$$C_p \geq C_{p-1}, \qquad \lim_{p \to \infty} C_p = C_{\max}.$$

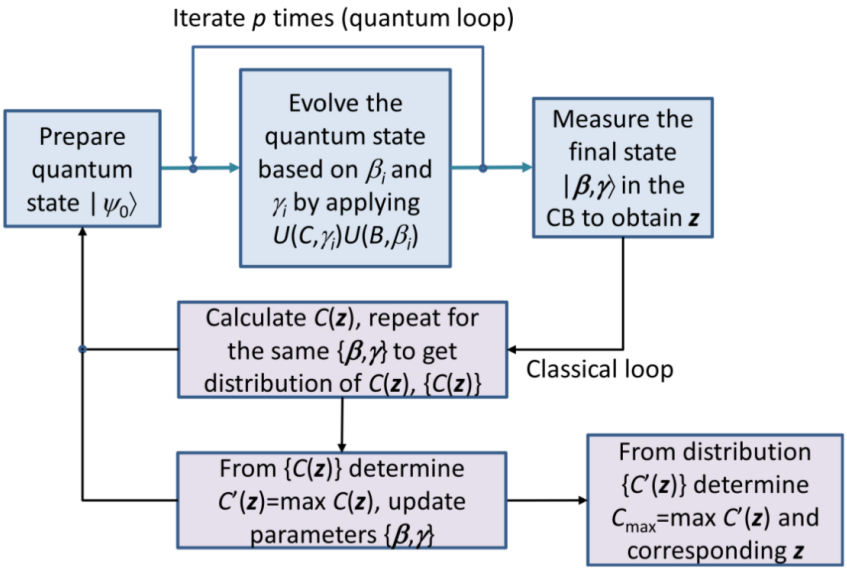


Figure 5.3: QAOA procedure [1].

Chapter 6

Application of QAOA to The Max-Cut Problem

6.1 Problem Statement

We now apply QAOA to the Max-Cut problem, which is one of the most important examples in combinatorial optimization and graph theory. Given an undirected graph $G = (V, E)$, the objective is to divide the set of vertices into two disjoint subsets, say S and S', in such a way that the number of edges connecting vertices in opposite subsets is as large as possible. The set of edges that cross from one subset to the other is called the *cut* and is denoted by δ.

The Max-Cut problem is interesting for two reasons. First, it has a simple graphical interpretation, which makes it a natural example for illustrating QAOA. Second, it is computationally hard in general, so it serves as a useful test case for quantum optimization algorithms. In many practical situations, one is not just interested in finding any partition, but rather the one that maximizes the number of separated edges. This makes Max-Cut a standard benchmark for both classical approximation algorithms and quantum variational methods.

6.2 Example of a Square Graph

Consider a square ring with $N = 4$ nodes. Since each node may be assigned to either of the two subsets, there are 2^N possible graph partitions in total. One convenient way to represent a partition is by using a 4-bit string. If a particular bit belongs to the set S, we assign the value 1; otherwise, we assign 0. In this way, every binary string corresponds to one possible division of the graph into two parts.

For a given edge $\langle j, k\rangle$, define the function $C_{\langle j,k\rangle}$ to return 1 when the two vertices j and k lie in different subsets, and 0 otherwise. In other words, the function checks whether that edge is cut by the partition. The total cost function is then the sum over all such edge contributions:

$$C = \sum_{\langle j,k\rangle} C_{\langle j,k\rangle}, \qquad C_{\langle j,k\rangle} = \frac{1}{2}(1 - z_j z_k),$$

where $z_i = +1$ if the ith node belongs to set S, and $z_i = -1$ otherwise. If the edge $\langle j, k\rangle$ is cut, then $z_j z_k = -1$, and the corresponding contribution becomes 1. If the two vertices lie in the same subset, then $z_j z_k = +1$, and the contribution becomes 0. Therefore, maximizing the cost function is equivalent to maximizing the number of cut edges.

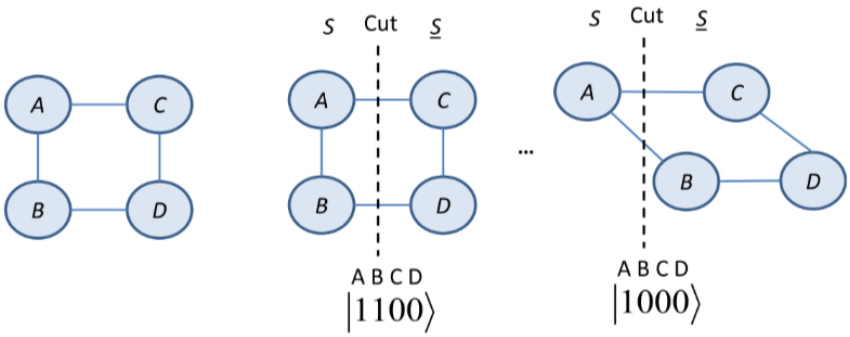


Figure 6.1: Max-Cut problem [1].

The objective function can again be represented as an operator:

$$C\left|z\right\rangle = \sum_{\langle i,k\rangle} C_{\langle i,k\rangle}(z)\left|z\right\rangle = C(z)\left|z\right\rangle.$$

This operator is diagonal in the computational basis, which makes it especially convenient for QAOA. Each basis state $|z\rangle$ represents a candidate partition, and the corresponding eigenvalue gives the number of edges cut by that partition.

The corresponding unitary evolution operator for C is

$$U(C,\gamma) = e^{-i\gamma C} = e^{-i\gamma \sum_{\langle j,k\rangle} C_{\langle j,k\rangle}} = \prod_{\langle j,k\rangle} U(C_{\langle j,k\rangle},\gamma), \tag{6.1}$$

where

$$U(C_{\langle j,k\rangle},\gamma) = e^{-i\gamma C_{\langle j,k\rangle}} = e^{-i\frac{\gamma}{2}I} e^{-i\frac{-\gamma}{2}Z_j Z_k}.$$

This form shows that the cost Hamiltonian can be implemented using two-qubit interaction terms. Hence, Max-Cut provides a direct and physically meaningful example of how a graph optimization problem can be encoded into the QAOA framework.

Chapter 7

Qiskit Implementation

In this chapter we will see the Qiskit Implementation of the MAX-CUT problem. We will deal with 2 variations of the problem stated in the section 6.2. The details of the program are given in the the Appendix A. Here we briefly discuss the findings and the success of the simulation on a Quantum Computer Simulator called *QISKIT_AER*.

7.1 The quantum circuit

As described above in section 5.3 and equation 5.14 we will apply the horizontal and vertical approximations as unitary rotation gates. In the circuit H stands for the Hadamard Gate , R_z for single qubit rotation about Z-axis and the $+$ stands for a controlled Not gate [2].

7.1.1 Case-1: n = 4 and edges = 4

In this case (refer to the figure 7.1) we have the nodes connected in a cyclic fashion $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 0$. The numbers on the circle identify the nodes and the respective numbers also identify with the respective qubits. For example the node 0 is denoted by q_0 in the circuit 7.2. The circuit initializes the system by applying a Hadamard transform[1] on all the qubits. Then

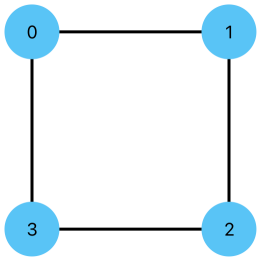


Figure 7.1: case 1 $|0000\rangle$ [3]

[1]The Hadamard transform creates equal superpositions of computational basis states using the Hadamard gate

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}.$$

refer to eqn 5.5 and also to [2]

depending on the edges in the graph it selectively entangles two nodes, applies the rotation gate of angle 2γ (along Z direction) and then reverts the entanglement. This is then followed by a rotation gate with rotation angle of 2β (along X direction). This entire process is repeated p times, where p is the number of trotterization steps. If we increase p we expect to see an increase in accuracy.

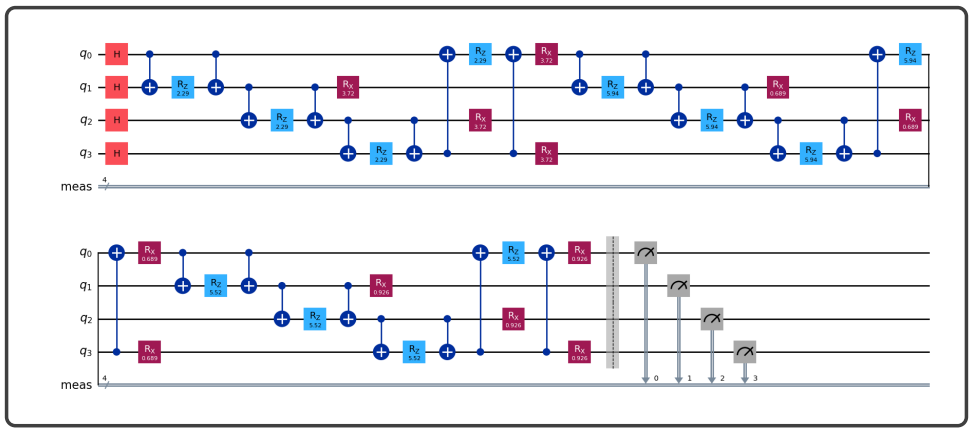


Figure 7.2: Circuit for QAOA application on a square grid with p = 3 [3].

We run this circuit with choosing initial parameters randomly and then using the *'COBYLA'* method to optimize the cost function gradient free. Figure 7.2 shows the circuit for $p = 3$ after the optimization is completed. Thus using Quantum Variational Circuit along with a Classical Optimizer we reach at the final set of parameters that yield us the solution to our MAX-CUT problem in terms of bit strings.

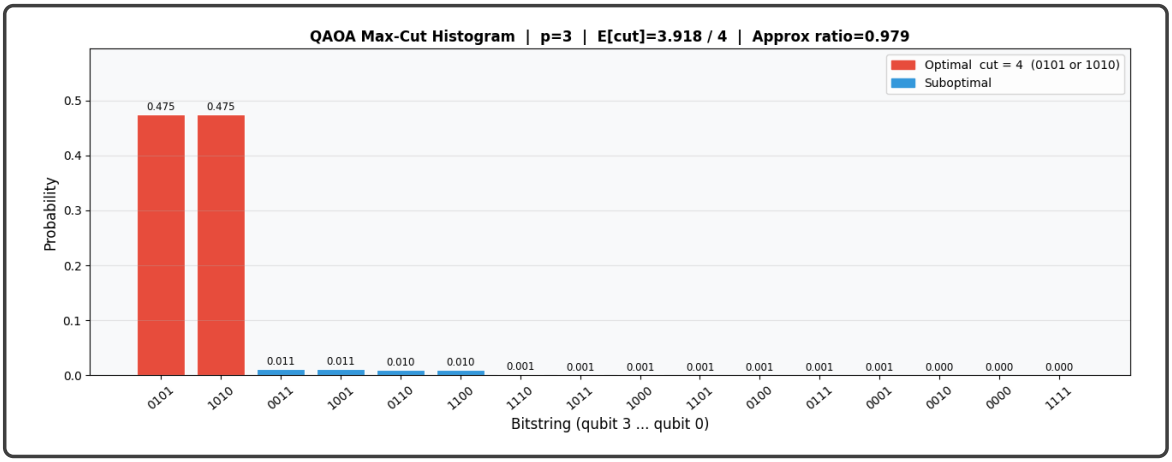


Figure 7.3: Results of 8192 shots for case 1, p = 3. [3]

We see in the fig 7.3 the bitstrings $|0101\rangle$ and $|1010\rangle$ have the maximum probability. These final states correspond to the graphs shown in fig 7.4. These two configurations have a cut value of 4 which is the maximum that can be achieved. For detailed analysis of accuracy and p value dependence refer to Appendix A.

(a) $|0101\rangle$ (b) $|1010\rangle$

Figure 7.4: Solutions to case 1 of Max Cut problem

7.1.2 Case-2: n = 4 and edges = 6

We study a variation of the graph by adding two new edges between 0 and 2 & 1 and 3. By addition of these new edges our circuit complexity has increased. Keeping $p = 3$ we repeat the

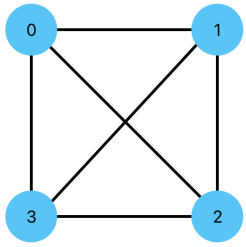


Figure 7.5: Case 2 $|0000\rangle$

same procedure as in case 1.

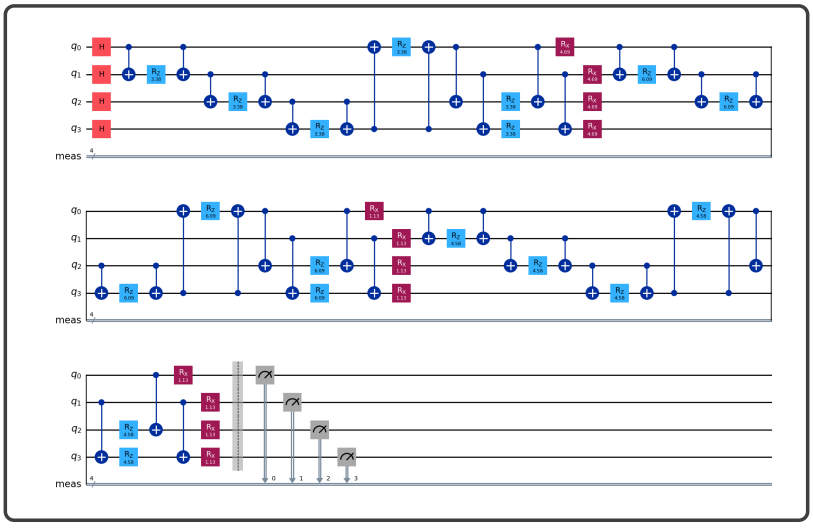


Figure 7.6: Circuit for QAOA application on a square grid with n=4 and edges = 6 and p = 3 [3].

We see this time the probability distribution plot has 6 peaks in fig 7.7 .
This means there are 6 solutions to this graph.

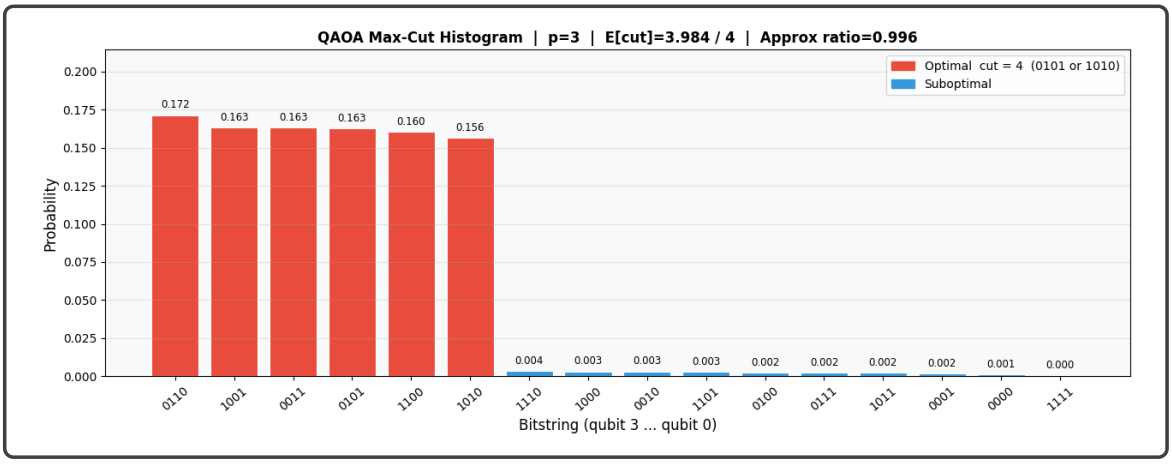
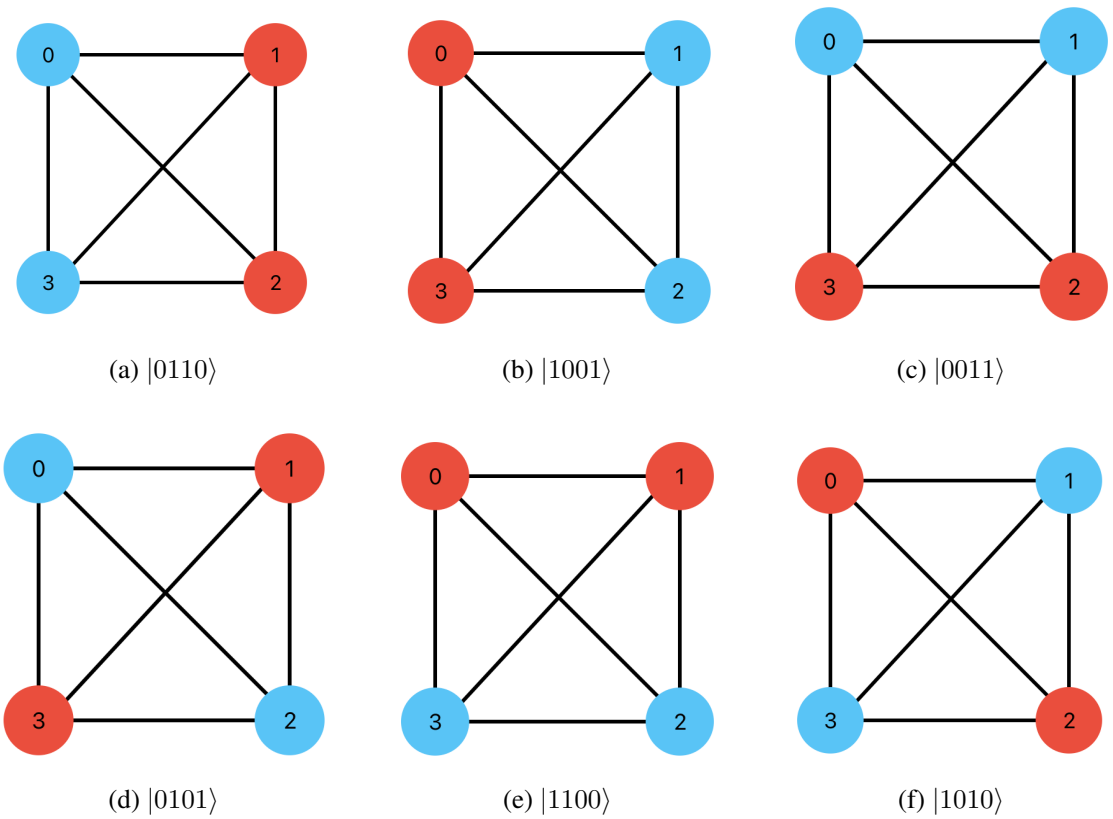


Figure 7.7: Results of 8192 shots for case 2, p = 3. [3]

The solutions $|0110\rangle$, $|1001\rangle$, $|0011\rangle$, $|0101\rangle$, $|1100\rangle$, and $|1010\rangle$ are shown in Fig. 7.8.



(a) $|0110\rangle$ (b) $|1001\rangle$ (c) $|0011\rangle$

(d) $|0101\rangle$ (e) $|1100\rangle$ (f) $|1010\rangle$

Figure 7.8: Solutions to case 2 of the Max-Cut problem.

All the solution correspond to a cut value of 4.

7.2 Results

Using the Quantum Approximate Optimization Algorithm we found the maximum cut value and the configuration of a square grid for two different graphs.

Case-1: n = 4 and edges = 4 had a max cut value of 4 and the two solutions to this were $|0101\rangle$ ***and*** $|1010\rangle$***. The Approx ratio = 0.979.*** It says the accuracy of the final evaluated expectation value of cut after classical optimization. it is defined as Expectation[cut]/Actual[cut]. As actual max cut is known to be 4 for this simple graph, we get Approx ratio to be : E[cut]/4.

Case-2: n = 4 and edges = 6 had a max cut value of 4 as well and there were 6 solutions: $|0110\rangle$, $|1001\rangle$, $|0011\rangle$, $|0101\rangle$, $|1100\rangle$***, and*** $|1010\rangle$***. Approx ratio = 0.996.***

The values of parameters and Approx ratio stated in this report are generic trend values. The values are not reproducible. The quantum circuit being probabilistic, the stated values give a heuristic picture of the solution and are not absolute in nature.

For a graph with large n value and large number of edges, this problem becomes very difficult to solve using a classical methods and QAOA's true potential is brought into use. The study of order of operations and computation time were beyond the scope of this study and have not been included. Future prospects of this study include exploring different optimization methods during the classical processing of the data and scalability studies of the quantum processors.

Chapter 8

Summary

This report presented a conceptual and mathematical introduction to the Quantum Approximate Optimization Algorithm by connecting ideas from machine learning, statistical physics, and quantum computation. The discussion began with a brief overview of machine learning and its common tasks, in order to emphasize that many practical problems can ultimately be phrased as optimization problems over structured data. Concepts such as feature vectors, similarity measures, and loss functions were introduced to build the broader context in which variational and optimization-based methods operate.

The report then moved to the Ising model, which provides a natural bridge between physics and optimization. By interpreting spin configurations as binary decision variables, the search for the ground state of the Ising Hamiltonian was shown to be equivalent to solving a combinatorial optimization problem. This connection was further extended through the relation between the Ising formulation and QUBO problems, demonstrating why many discrete optimization tasks can be encoded into a form suitable for quantum algorithms.

On this foundation, the report introduced the basic principles of quantum dynamics relevant to optimization, including time evolution, transverse field Hamiltonians, adiabatic quantum computing, and quantum annealing. These ideas motivated the study of QAOA as a variational and discretized alternative to adiabatic evolution. In QAOA, the system evolves through alternating applications of a cost Hamiltonian and a mixer Hamiltonian, with the corresponding angles treated as tunable parameters optimized by a classical routine. The Max-Cut problem served as the main example throughout the report. It was shown how a graph partitioning problem can be translated into a cost operator that is diagonal in the computational basis, making it especially suitable for QAOA. The report described how the associated unitary operators are constructed and why the problem provides a clear illustration of how combinatorial optimization is embedded into a quantum circuit model.

Finally, a Qiskit-based simulation was discussed for two four-node graph instances. The results showed that the algorithm successfully concentrates probability on the optimal bit strings corresponding to the maximum cuts, thereby illustrating the practical workflow of QAOA: parameterized circuit preparation, classical optimization, and measurement-based identification of good solutions. Overall, the report demonstrated that QAOA is not only a useful variational algorithm for near-term quantum devices, but also an important framework for understanding how physical Hamiltonians, optimization theory, and quantum circuit design come together in quantum machine learning and quantum optimization.

At the same time, the study has several important limitations. The analysis is restricted to small graph instances simulated on a classical backend, so it does not establish any practical

quantum advantage over classical optimization methods. The results are therefore illustrative rather than conclusive, and they mainly demonstrate the logic of the algorithm rather than its large scale performance. In addition, only a limited set of graph examples, parameter choices, and optimization settings are considered, so questions of scalability, hardware noise, execution cost, and performance on realistic network design instances remain outside the scope of this report. These limitations should be kept in mind when interpreting the findings, but they do not reduce the value of the study as a conceptual and methodological introduction to QAOA.

Appendix A

Jupyter Notebook for Max-Cut

The implementation used in this study is contained in the *jupyter notebook* which is uploaded in the github repository of the username: *AshisNayak-quantum* that can be accessed from the reference [3]. The notebook demonstrates, step by step, how the Max-Cut problem on a small graph is encoded and solved using the Quantum Approximate Optimization Algorithm in Qiskit.

Parameters

The following parameters are to be defined by the user before initializing the routine:

- n is the number of nodes in the graph (In this example it is 4). It defines the number of qubits required for the Quantum Circuit (also the number of Quantum Channel).
- p is the circuit depth or the number of trotterization steps. One can expect the accuracy of the circuit to increase with increase in p and so does the run time.
- γ and β are the rotation angles, there will be $2p$ rotation angle parameters. p different γ and p different β.
- *COBYLA* is the classical optimizer model used in the code. It is a gradient free optimizer which reduces computation cost. Other optimizers can be used upon increase in p or n values or to incorporate noise of quantum channel into account. But for the scope of this project *COBYLA* suffices.
- *shots* : it is number of times the final circuit with estimated parameters is run to get the optimum bitstring probabilities. *The law of large numbers* [4] suggests as $shots \to \infty$ the probability distribution for optimum bitstring tends to actual value. Keeping *shots* as high as 8192 is safe to say the outcome of this many runs will give near accurate description for the distribution of the bitstrings.

Different parameters can be varied in the circuit for the study of convergence for the optimal values in the circuit. Our goal is to find rotation angles that give the optimal circuit which simulates the annealing process for optimal bitstring.

For example the optimum rotational parameters obtained for $p = 5$ are:
Best γ (list of γ_k, $k \in \{1, 2, 3, 4, 5\}$) : [1.3423 2.9787 2.3334 2.0564 0.5022]
Best β (list of β_k, $k \in \{1, 2, 3, 4, 5\}$) : [0.6082 0.3291 2.8746 2.0258 2.8874]
These converged values were found after 750 iterations of "*COBYLA*" method. To define the

accuracy of the final optimized circuit we find the *Approx ratio*. It is the ratio of average cut value achieved for the total shots to the analytical expected cut value for optimum bitstring (which is 4). For $p \geq 4$ we get an *Approx ratio* of 1. For $p = 3$ we get an *Approx ratio* of 0.979.

Comparative study

For the case 1 (refer to 7.1.1) the following plots show the contribution of optimal bitstrings to the probability distribution vs chosen p (circuit depth) for the optimized circuit:

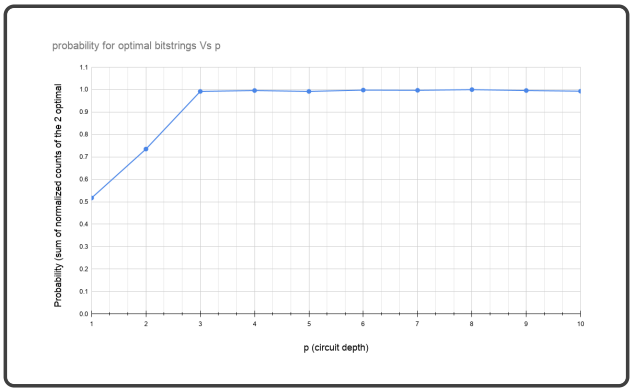
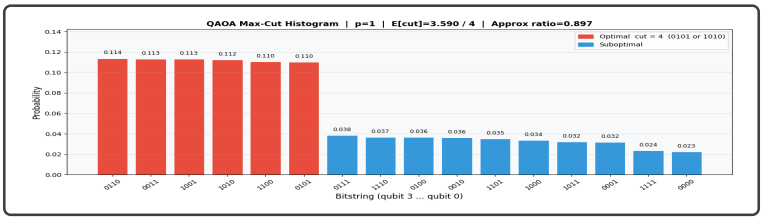


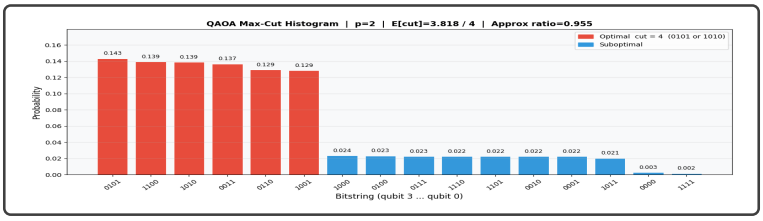
Figure A.1: Graph for probability vs p value, for p between 1 and 10

Fig A.1 shows that for $p \geq 3$ we get optimum expectation values for optimum bitstrings. Thus for running a QAOA circuit for this graph the optimum circuit depth is p equal to 3 or 4.

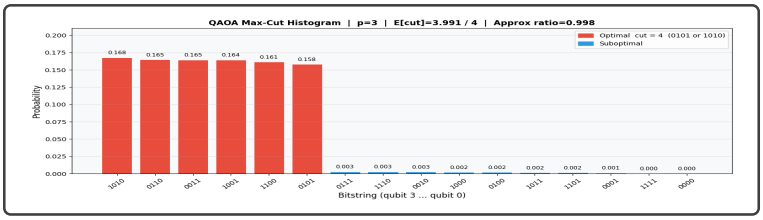
Similar study of case 2 (refer to 7.1.2) shows that optimum value of cut value is again achieved for $p = 4$.



$p = 1$



$p = 2$



$p = 3$

Figure A.2: Probability distributions for case 2 (7.1.2) at circuit depths $p = 1$, $p = 2$, and $p = 3$.

Bibliography

[1] Ivan B. Djordjevic. *Quantum Information Processing, Quantum Computing, and Quantum Error Correction: An Engineering Approach*. Elsevier, 2 edition, 2021.

[2] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, Cambridge, 1st edition, 2000.

[3] Ashis Nayak. Quantum-machine-learning-qaoa. https://github.com/AshisNayak-quantum/Quantum-Machine-Learning-QAOA, 2026. Accessed: 2026-05-03.

[4] Frederik Michel Dekking, Cornelis Kraaikamp, Hendrik Paul Lopuhaä, and Ludolf Erwin Meester. *A Modern Introduction to Probability and Statistics: Understanding Why and How*. Springer, London, 2005.